DIASCORA

Diaspora Community Web Platform

Software Requirements Document

MVP Edition — Module Implementation Plan & Micro-Task Breakdown

Field	Detail
Document Version	3.0 — MVP Edition
Platform Name	Diascora
Date	28 February 2026
Status	Draft
Backend Stack	Laravel 11 (PHP 8.3)
Frontend Stack	Inertia.js + Vue 3 + Tailwind CSS
Forum Engine	Discourse (discourse.org, self-hosted Docker)
Classification	Confidential

1. Executive Summary

Diascora is a community web platform that connects diaspora populations — people sharing the same country of origin who now reside in a common host country. The platform organises these groups into dynamic country-pair communities (e.g. Nigerians in Rwanda — ng-in-rw) and provides each community with tools for peer-to-peer currency exchange, a structured forum for fixed community posts and service requests, and a notifications system to keep members informed.

This document defines the MVP scope, functional and non-functional requirements, the full Laravel-based technology stack, and a micro-task implementation plan for every module. The MVP deliberately narrows scope to deliver a working, deployable product quickly, with a clear path to the full feature roadmap in subsequent releases.

2. MVP Scope

2.1 Included in MVP

- Google OAuth 2.0 SSO (single provider via Laravel Socialite)
- Two-step onboarding: home country + resident country (no phone/OTP in MVP)
- Dynamic country-pair community pages
- P2P Exchange Request Board — anonymous posting, interest reveal, 24-hour expiry
- Community Forum powered by Discourse (discourse.org, self-hosted) with DiscourseConnect SSO
- Fixed/pinned community posts and service-request threads inside Discourse
- Email notifications (Mailgun/Postmark via Laravel Mail)
- PWA push notifications (Web Push API via minishlink/web-push)
- User notification preference management (per section, per channel)
- Admin-granted Trusted Badge — platform admins can manually award a trust badge to credible members

2.2 Deferred to Post-MVP

- Facebook and Twitter/X SSO
- Mobile phone verification and OTP
- WhatsApp notification channel
- Community services directory (separate listings module)
- Tips & government news section (admin-curated articles)
- Automated milestone cards and activity-driven badge engine (manual admin badge is in MVP)
- User-initiated verification (school, work, address) with document upload
- Peer profile ratings and comments
- Restriction and moderation admin panel
- Native mobile apps (iOS / Android)

3. User Personas

Persona	Description	MVP Needs
Community Member	Onboarded diaspora user	Post/browse exchange requests, participate in forum
Guest / Visitor	Non-registered visitor	Read public community info and forum (read-only)
Community Admin	Moderator of a community	Pin forum posts, remove violating content (via Discourse)
Platform Admin	Diascora superadmin	Manage communities, approve Discourse categories, monitor platform

4. Functional Requirements

Ref	Module	Requirement	Must / Should	Type
FR-01	Auth	Users authenticate exclusively via Google OAuth 2.0 in MVP	Must	Functional
FR-02	Auth	Authenticated sessions managed with Laravel Sanctum (HttpOnly cookie)	Must	Security
FR-03	Onboarding	System captures home country and resident country before granting access	Must	Functional
FR-04	Onboarding	Middleware blocks access to app until onboarding is marked complete	Must	Functional
FR-05	Community	Community pages are auto-generated per unique country pair (slug: origin-in-residence)	Must	Functional
FR-06	Exchange	Users post exchange requests anonymously; identity hidden from board listing	Must	Functional
FR-07	Exchange	Contact details revealed only when another member clicks 'Show Interest'	Must	Functional
FR-08	Exchange	Unmatched requests auto-expire after 24 hours via Laravel scheduler	Must	Functional
FR-09	Exchange	Poster notified on expiry with option to repost	Must	Functional
FR-10	Forum	Discourse forum embedded per community with per-community category grouping	Must	Functional
FR-11	Forum	Admin can pin fixed/announcement posts per community	Must	Functional
FR-12	Forum	Members can create 'Service Request' threads under a dedicated tag	Must	Functional

Ref	Module	Requirement	Must / Should	Type
FR-13	Forum	Diascora SSO authenticates users to Discourse via DiscourseConnect (no separate login)	Must	Security
FR-14	Notifications	Email notifications sent for exchange events (expiry, interest received)	Must	Functional
FR-15	Notifications	PWA push notifications supported for exchange and forum events	Should	Functional
FR-16	Notifications	Users can toggle email and push per notification section independently	Must	Functional
FR-17	Trusted Badge	Only platform admins can award or revoke the Trusted Badge on a user profile	Must	Functional
FR-18	Trusted Badge	Trusted Badge is visibly displayed on user profiles and exchange request cards	Must	Functional
FR-19	Trusted Badge	User receives email and push notification when Trusted Badge is awarded	Should	Functional

5. Non-Functional Requirements

Category	Requirement	Target / Standard
Performance	Server response time under typical load	< 300ms TTFB (P95)
Performance	Full page load (LCP)	< 2.5 seconds on 4G
Security	All traffic encrypted in transit	TLS 1.2+ enforced
Security	OAuth tokens never stored in localStorage	HttpOnly cookies only
Security	CSRF protection on all state-changing endpoints	Laravel CSRF middleware
Scalability	Queue workers handle spike in notifications	Laravel Horizon + Redis
Availability	Platform uptime SLA	99.5% monthly
Compliance	User data protection	GDPR-aligned; right to deletion
Accessibility	Web accessibility standard	WCAG 2.1 Level AA
Mobile	Fully responsive across breakpoints	320px minimum viewport width
PWA	Installable PWA with offline shell	Lighthouse PWA score > 90

6. Technology Stack — Laravel Ecosystem

Diascora is built entirely within the Laravel ecosystem, ensuring cohesion, a rich package ecosystem, and ease of deployment. The following table details the canonical stack for MVP.

Layer / Concern	Technology	Purpose
Backend Framework	Laravel 11 (PHP 8.3)	Core application, routing, ORM, auth, mail, queues, scheduler
ORM & Database	Eloquent ORM + MySQL 8 / PostgreSQL 16	Data modelling, relationships, migrations
Authentication	Laravel Socialite (Google) + Sanctum	OAuth SSO, API token / session management
Frontend	Inertia.js + Vue 3 + Tailwind CSS 3	SPA-like UX without API overhead; SSR via Inertia
Build Tool	Vite (Laravel Vite Plugin)	Asset bundling, HMR in development
Queue / Workers	Laravel Horizon + Redis	Background jobs: notifications, expiry, push dispatch
Task Scheduling	Laravel Scheduler (cron)	Hourly exchange expiry sweep, daily digest emails

Layer / Concern	Technology	Purpose
Email	Laravel Mail + Mailgun / Postmark driver	Transactional email delivery
Push Notifications	minishlink/web-push (PHP) + Service Worker	VAPID-based Web Push for PWA subscribers
Forum Engine	Discourse (open-source, Ruby on Rails)	Self-hosted community forum with DiscourseConnect SSO bridge
Forum SSO	DiscourseConnect SSO protocol (built-in Discourse feature)	Single sign-on from Diascora to Discourse — no custom plugin needed
Caching	Redis + Laravel Cache facade	Query caching, rate limiting, session store
File Storage	Laravel Filesystem + S3 / Cloudflare R2	User avatars, future document uploads
Testing	Pest PHP + Laravel TestCase + Playwright	Unit, feature, and E2E tests
Deployment	Laravel Forge + Nginx + MySQL + Supervisor	Production server management and process supervision
CI / CD	GitHub Actions	Automated test runs, code style checks, deployments

7. System Architecture Overview

Diascora follows a monolithic Laravel application architecture with Inertia.js bridging the backend and Vue 3 frontend. This maximises development velocity for the MVP while remaining straightforward to extract into services post-MVP. The forum runs as a separate Discourse installation on a subdomain (forum.diascora.app) and is connected to Diascora via the built-in DiscourseConnect SSO protocol — no custom plugin required.

7.1 Request Flow

- Browser → Nginx → Laravel App Server (PHP-FPM)
- Laravel handles routing, business logic, Eloquent queries
- Inertia.js renders Vue 3 components with server-supplied props (no separate API)
- Background jobs dispatched to Redis queue; processed by Laravel Horizon workers
- Scheduled commands run via cron every minute, executing due tasks (expiry sweep, etc.)

7.2 Forum Integration Flow

- User authenticated in Diascora (Sanctum session active)
- User clicks 'Forum' tab → Diascora generates a signed JWT (user_id, display_name, email, community_tag)
- Browser redirected to forum.diascora.app/auth/sso?token=<JWT>
- Discourse validates the HMAC-signed payload, auto-creates or updates the Discourse user, and issues a Discourse session cookie
- User lands in Discourse pre-authenticated, scoped to their community category

8. Key Database Tables

Table	Key Columns
users	id, google_id, name, email, avatar_url, home_country, resident_country, onboarding_complete, created_at
countries	code (ISO 3166-1), name, flag_emoji, dial_code
communities	id, slug (ng-in-rw), origin_country, resident_country, member_count, discourse_category_id, created_at
exchange_requests	id, community_id, poster_user_id, currency_from, currency_to, amount, rate, notes, status, expires_at
exchange_interests	id, request_id, interested_user_id, revealed_at, created_at
badges	id, user_id, badge_type (trusted), awarded_by (admin user_id), reason, awarded_at, revoked_at
push_subscriptions	id, user_id, endpoint, p256dh, auth, user_agent, created_at

Table	Key Columns
notification_preferences	id, user_id, section (exchange|forum), email_enabled, push_enabled
notifications (Laravel)	id, type, notifiable_type, notifiable_id, data (JSON), read_at, created_at

9. Module Implementation Plan & Micro-Task Breakdown

Each module is broken into atomic, independently implementable tasks. Effort estimates assume a mid-senior Laravel/Vue developer. All backend tasks include writing the corresponding migration, model, controller, routes, and form request validation unless otherwise noted.

M01: Authentication — Google SSO

Description	Google OAuth 2.0 login only for MVP. Handled via Laravel Socialite. JWT sessions managed with Laravel Sanctum.

Priority	Critical

Task ID	Task Description	Effort	Type	Phase
M01-T01	Register Google OAuth app; configure client ID & secret in .env	1h	DevOps	MVP
M01-T02	Install & configure Laravel Socialite (google driver)	1h	Backend	MVP
M01-T03	Create SocialAuthController: redirect() and callback() methods	2h	Backend	MVP
M01-T04	Upsert User record on successful OAuth callback (name, email, avatar, google_id)	2h	Backend	MVP
M01-T05	Issue Laravel Sanctum API token on login; store in HttpOnly cookie	2h	Backend	MVP
M01-T06	Build login page UI — single 'Continue with Google' button (Blade/Inertia)	3h	Frontend	MVP
M01-T07	Implement logout route — revoke Sanctum token	1h	Backend	MVP
M01-T08	Protect all authenticated routes via auth:sanctum middleware	1h	Backend	MVP
M01-T09	Write feature tests for login, callback, and logout flows	3h	Testing	MVP

M02: User Onboarding

Description	Multi-step onboarding capturing home country and resident country. Mobile number (with OTP) is deferred post-MVP. Profile stored immediately after SSO.

Priority	Critical

Task ID	Task Description	Effort	Type	Phase
M02-T01	Create users table migration: home_country, resident_country, onboarding_complete	1h	Backend	MVP
M02-T02	Build OnboardingController with step 1 (home country) and step 2 (resident country)	3h	Backend	MVP
M02-T03	Build onboarding UI — searchable country dropdown (Step 1)	3h	Frontend	MVP
M02-T04	Build onboarding UI — resident country dropdown (Step 2)	2h	Frontend	MVP
M02-T05	Add OnboardingMiddleware — redirect incomplete users to onboarding	2h	Backend	MVP
M02-T06	Validate and persist country selections; set onboarding_complete = true	2h	Backend	MVP
M02-T07	Seed countries table from ISO 3166-1 alpha-2 list	1h	Backend	MVP
M02-T08	Write feature tests for onboarding flow and middleware redirect	2h	Testing	MVP

M03: Community Pages

Description	Dynamic pages per country-pair (e.g. /community/ng-in-rw). Auto-generated from user home/resident country data. Tabs link to Exchange and Forum sections.

Priority	Critical

Task ID	Task Description	Effort	Type	Phase
M03-T01	Create communities table: origin_country, resident_country, slug (ng-in-rw), member_count	1h	Backend	MVP
M03-T02	Auto-create Community record on onboarding if it does not exist	2h	Backend	MVP
M03-T03	Build CommunityController: show() resolves slug to community	2h	Backend	MVP
M03-T04	Build community page layout (Blade/Inertia): tabs — Exchange, Forum, Members	5h	Frontend	MVP
M03-T05	Build community discovery/search page — list all active communities	3h	Frontend	MVP
M03-T06	Build community header component: flag icons, member count, join status	3h	Frontend	MVP
M03-T07	Implement access rules: guests see community info; only members can post	2h	Backend	MVP

Task ID	Task Description	Effort	Type	Phase
M03-T08	Write route model binding tests and access-control tests	2h	Testing	MVP

M04: P2P Exchange Request Board

Description	Anonymous exchange request board per community. Poster identity hidden until interest is shown. Requests auto-expire after 24 hours via Laravel scheduler. Notifications via email and PWA push.

Priority	Critical

Task ID	Task Description	Effort	Type	Phase
M04-T01	Create exchange_requests table: community_id, poster_user_id, currency_from, currency_to, amount, rate, notes, status, expires_at	2h	Backend	MVP
M04-T02	Create exchange_interests table: request_id, interested_user_id, revealed_at	1h	Backend	MVP
M04-T03	Build ExchangeRequestController: index(), store(), show(), close()	4h	Backend	MVP
M04-T04	Build 'Post Exchange Request' form UI — currency pair, amount, rate, optional note	4h	Frontend	MVP
M04-T05	Build exchange board UI — card list with anonymous display (no poster name shown)	5h	Frontend	MVP
M04-T06	Implement 'Show Interest' action — create exchange_interests record	2h	Backend	MVP
M04-T07	Build contact reveal flow — poster sees interested party contact after interest shown	3h	Backend	MVP
M04-T08	Build interested party list on poster's My Requests view	3h	Frontend	MVP
M04-T09	Implement ExchangeExpiry scheduled job (Laravel scheduler, runs hourly)	3h	Backend	MVP
M04-T10	Dispatch ExchangeExpiredNotification (Mail + Push) to poster on expiry	3h	Backend	MVP
M04-T11	Build 'Repost' action from expired request — clone with new expiry	2h	Backend	MVP
M04-T12	Build 'My Exchange Requests' profile section with status badges	3h	Frontend	MVP
M04-T13	Write feature tests for full request lifecycle (create → interest → reveal → expire)	5h	Testing	MVP

M05: Community Forum (Discourse)

Description	Open-source forum powered by Discourse (discourse.org), self-hosted per platform. Supports pinned/fixed posts (announcements) and service request threads organised by community category. Integrated with Diascora via Discourse Connect (DiscourseConnect SSO protocol).

Priority	High

Task ID	Task Description	Effort	Type	Phase
M05-T01	Provision Discourse on dedicated subdomain (forum.diascora.app) using official Docker image	3h	DevOps	MVP
M05-T02	Configure Discourse: site name, admin account, SMTP for forum emails	2h	DevOps	MVP
M05-T03	Enable DiscourseConnect (formerly SSO) in Discourse admin settings; set shared secret	1h	DevOps	MVP
M05-T04	Build DiscourseConnect endpoint in Laravel: /forum/sso — generate signed payload (nonce, email, username, name, avatar)	4h	Backend	MVP
M05-T05	Validate Discourse nonce and HMAC signature on return callback; create/update local sso_session	3h	Backend	MVP
M05-T06	Create per-community Discourse Categories (ng-in-rw, gh-in-de) via Discourse REST API on community creation	4h	Backend	MVP
M05-T07	Auto-provision community category and sub-categories on Diascora community creation event	2h	Backend	MVP
M05-T08	Create two sub-categories per community: Announcements (pinned) and Service Requests	1h	Content	MVP
M05-T09	Post default pinned 'Welcome & Community Rules' topic per community category via Discourse API	2h	Backend	MVP
M05-T10	Embed Discourse community forum inside Diascora via redirect tab (link to scoped category URL)	3h	Frontend	MVP
M05-T11	Apply Diascora brand colours to Discourse (custom CSS via Discourse Theme Component)	4h	Frontend	MVP
M05-T12	Configure Discourse Trust Levels and moderation settings; restrict posting to logged-in users	2h	DevOps	MVP
M05-T13	Sync Diascora ban status to Discourse via API (suspend user on ban)	3h	Backend	MVP
M05-T14	Write integration tests: SSO login flow, category creation, post visibility, ban sync	4h	Testing	MVP

M06: Admin-Granted Trusted Badge

Description	A single Trusted Badge manually awarded by platform admins to recognise credible, well-behaved members. Badge is visible on user profiles and exchange posts. No automated criteria in MVP — admin discretion only. Foundation for the full trust/milestone system in a future release.

Priority	High

Task ID	Task Description	Effort	Type	Phase
M06-T01	Create badges table: id, user_id, badge_type (trusted), awarded_by (admin user_id), reason, awarded_at	1h	Backend	MVP
M06-T02	Build BadgeController: index() lists all badge holders, store() awards badge, destroy() revokes badge	3h	Backend	MVP
M06-T03	Add admin-only gate to all badge endpoints using Laravel Gates / Spatie permission (role: platform-admin)	1h	Backend	MVP
M06-T04	Build admin Badge Management UI: searchable user list, Award Badge button, optional reason field	4h	Frontend	MVP
M06-T05	Build badge revocation flow in admin UI with confirmation modal	2h	Frontend	MVP
M06-T06	Display Trusted Badge icon (shield/tick) on user profile page with awarded date tooltip	2h	Frontend	MVP
M06-T07	Display compact Trusted Badge indicator on exchange request cards next to anonymous poster label	2h	Frontend	MVP
M06-T08	Dispatch TrustedBadgeAwardedNotification (email + push) to user when badge is granted	2h	Backend	MVP
M06-T09	Add badge history log entry via spatie/laravel-activitylog on award and revoke	1h	Backend	MVP
M06-T10	Write feature tests: award, revoke, non-admin access denial, notification dispatch	3h	Testing	MVP

M07: Email & PWA Push Notifications

Description	MVP notifications delivered via email (Laravel Mail + Mailgun/Postmark) and browser push (Web Push API via PWA). Users subscribe per section and manage channel preferences.

Priority	High

Task ID	Task Description	Effort	Type	Phase
M07-T01	Configure Laravel Mail with Mailgun or Postmark driver in .env	1h	DevOps	MVP
M07-T02	Create Mailable classes: ExchangeExpiredMail, InterestReceivedMail, TrustedBadgeAwardedMail, WelcomeMail	4h	Backend	MVP
M07-T03	Design branded email templates (Blade components, responsive HTML)	5h	Frontend	MVP
M07-T04	Implement PWA manifest.json and service worker (sw.js) for Diascora	4h	Frontend	MVP
M07-T05	Implement Web Push subscription flow — request permission, store PushSubscription in DB	4h	Frontend	MVP
M07-T06	Create push_subscriptions table: user_id, endpoint, p256dh, auth, user_agent	1h	Backend	MVP
M07-T07	Install minishlink/web-push PHP library; build PushNotificationService	3h	Backend	MVP
M07-T08	Create notification_preferences table: user_id, section, email_enabled, push_enabled	1h	Backend	MVP
M07-T09	Build Notification Preferences UI — toggle email/push per section	4h	Frontend	MVP
M07-T10	Build NotificationPreferencesController: show() and update()	2h	Backend	MVP
M07-T11	Integrate preferences check inside notification dispatch logic	2h	Backend	MVP
M07-T12	Build in-app notification bell with unread count (DB notifications)	4h	Frontend	MVP
M07-T13	Queue all notifications via Laravel Horizon (Redis-backed queue)	2h	Backend	MVP
M07-T14	Write tests for mail dispatch, push delivery, and preference filtering	3h	Testing	MVP

10. MVP Implementation Phases

Six focused sprints (two weeks each) deliver a fully functional MVP. Each sprint produces working, deployable software.

Sprint	Timeline	Modules	Deliverable
Sprint 1	Weeks 1–2	M01: Auth + M02: Onboarding	Users can sign in with Google and complete country onboarding
Sprint 2	Weeks 3–4	M03: Community Pages	Community pages live, auto-created, and accessible by slug
Sprint 3	Weeks 5–7	M04: P2P Exchange Board	Full exchange lifecycle: post, interest, reveal, expire, repost
Sprint 4	Weeks 8–9	M05: Forum (Discourse)	Discourse running, DiscourseConnect SSO live, per-community categories, pinned posts
Sprint 5	Week 10	M06: Trusted Badge	Admins can award/revoke Trusted Badge; badge displayed on profiles and exchange cards
Sprint 6	Weeks 11–12	M07: Notifications	Email + PWA push live, preferences UI, notification bell, badge award notifications
Sprint 7	Weeks 13–14	QA + Performance + Launch	Full regression, load testing, production deploy on Laravel Forge

11. Discourse Forum — Integration Detail

11.1 Why Discourse

Discourse is the world's most widely used open-source forum platform (MIT licence), trusted by companies like GitHub, Figma, and Mozilla. It ships with a powerful REST API, a built-in SSO protocol (DiscourseConnect), granular trust levels, and robust moderation tools — all of which Diascora leverages without writing custom plugins. It runs on its own Docker container alongside the Laravel app.

11.2 Forum Structure Per Community (Discourse Categories)

- Each community (ng-in-rw) gets a dedicated Discourse Category auto-created via the Discourse REST API (POST /categories) when the community is first initialised in Diascora.
- Each community tag has two mandatory sub-tags:
  - Announcements — for pinned/fixed posts from admins (community rules, welcome message, government notices)
  - Service Requests — for members to post requests like 'looking for a Nigerian barber in Kigali'
- Category security settings in Discourse restrict visibility: only members of that community can see and post in their category.

11.3 SSO Bridge Implementation

DiscourseConnect eliminates a separate Discourse registration. When a Diascora user navigates to the Forum tab:

- Diascora backend generates a short-lived JWT (TTL: 5 minutes) signed with APP_KEY, containing: user_id, email, display_name, avatar_url, community_tag.
- Discourse redirects browser to forum.diascora.app/session/sso with a nonce; Diascora/forum/sso receives this and builds a signed payload (nonce, email, username, name, avatar_url, add_groups).
- Diascora returns a HMAC-SHA256 signed redirect to forum.diascora.app/session/sso_login — no custom plugin needed; Discourse handles this natively.
- Discourse issues a session cookie and redirects user to their community category page.
- Subsequent visits skip DiscourseConnect flow if the Discourse session cookie is still valid.

12. Trusted Badge — Admin Grant Detail

12.1 Overview

The Trusted Badge is the MVP's single credibility signal. It is a manual, admin-only action with no automated criteria. Its purpose is to let platform admins recognise members who have demonstrated reliability, positive community behaviour, or real-world identity confirmation through informal channels during the early platform lifecycle.

12.2 Badge Display Rules

- A shield/checkmark icon appears on the user's profile page next to their display name.
- A compact badge indicator appears on every exchange request card attributed to a Trusted member (even though the poster is anonymous, 'Trusted Member' replaces 'Member' in the card label).
- Hovering/tapping the badge shows a tooltip: 'Trusted by Diascora — awarded [date]'.
- If a badge is revoked, the icon is removed immediately with no historical display in MVP.

12.3 Admin Workflow

- Admin navigates to /admin/badges — a paginated, searchable table of all platform users.
- Admin clicks 'Award Badge' on a user row, optionally enters an internal reason (not shown publicly), and confirms.
- System calls BadgeController@store, creates the badge record, dispatches TrustedBadgeAwardedNotification.
- To revoke, admin clicks 'Revoke' — soft-deletes the badge record (sets revoked_at), dispatches a revocation notification.
- All award and revoke events are logged via spatie/laravel-activitylog for audit trail.

12.4 Security & Access Control

The badge endpoints are protected by a dedicated Laravel Gate ('manage-badges') assigned only to the 'platform-admin' role via spatie/laravel-permission. Any non-admin attempt to access award or revoke endpoints returns a 403 Forbidden. There is no self-service badge application in MVP.

13. Risks & Mitigations

Risk	Severity	Mitigation
Discourse DiscourseConnect breaks on Discourse major version update	Low	DiscourseConnect is a stable, core Discourse feature — low breakage risk. Pin Docker image tag in MVP; schedule upgrades with regression suite
Exchange posts enabling informal fraud or scams	High	Discourse community flagging; rate-limit exchange posts to 3/day per user in Laravel; future Trusted Badge gating
Admin abusing Trusted Badge grant (bias, favouritism)	Medium	All badge actions logged via activitylog; future release adds multi-admin approval and public badge criteria
Google OAuth policy changes or app review delays	Medium	Submit OAuth app early; maintain detailed OAuth consent screen and privacy policy before review
Web Push not supported on iOS Safari < 16.4	Medium	Gracefully degrade — email remains primary; show in-app bell for unsupported browsers
Discourse (Docker) and Laravel on same host causing resource contention	Medium	Run Discourse in its own Docker container with CPU/memory limits; monitor via Laravel Horizon + Discourse built-in dashboard
Email delivery to spam folders in target regions (Nigeria, Rwanda, etc.)	High	Configure SPF, DKIM, DMARC from day one; use dedicated sending IP via Mailgun/Postmark
GDPR deletion requests spanning both Diascora DB and Discourse DB	High	Implement DeleteUserJob: anonymise in Diascora DB + call Discourse API DELETE /admin/users/{id}.json; document in privacy policy

14. Appendix — Recommended Laravel Packages

Package	Version	Purpose
laravel/socialite	^5.0	Google OAuth 2.0 SSO
laravel/sanctum	^4.0	API token & session auth
inertiajs/inertia-laravel	^2.0	Inertia.js Laravel adapter
tightenco/ziggy	^2.0	Named Laravel routes in Vue
laravel/horizon	^5.0	Queue worker dashboard (Redis)
minishlink/web-push	^9.0	VAPID Web Push notifications (PHP)
spatie/laravel-permission	^6.0	Role & permission management (admin, member, moderator)
spatie/laravel-activitylog	^4.0	Audit trail for moderation actions
hirethunk/verbs	latest	Event sourcing for exchange lifecycle (optional)
pestphp/pest	^3.0	Elegant test framework (Pest PHP)
laravel/pint	^1.0	Laravel opinionated code style fixer